

A Coherent Market for Related Events

Technical Litepaper | Version 0.1

28 September 2026

MONAD PUBLIC TESTNET	TEST ASSETS ONLY

Abstract

Prediction markets usually divide related questions into separate venues. This design is operationally simple, but it discards a central object of interest: the joint distribution over outcomes. Separate prices can be individually plausible while failing to describe any coherent view of how events occur together. Separate liquidity also prevents a trade in one expression from informing the prices of logically related expressions.

Flurbo is an experimental information market for binary events and Boolean claims defined over those events. A pool represents a finite state space, maintains one logarithmic market scoring rule cost function, and prices individual and composed claims against the same inventory. A claim is a truth table over one to three selected events. The same representation supports individual answers, conjunction, disjunction, exactly-one, at-least-two, and other nonconstant Boolean payoffs. One terminal assignment settles all claims in a pool consistently.

The current Monad testnet application uses small event clusters with explicitly enumerated states, test AUSD collateral, immutable market rules, optimistic resolution, and a uniform treatment of void outcomes. A separately implemented factored engine uses bounded-width variable elimination to evaluate one global cost function without materializing all terminal states. The factored engine is locally tested and is not presented as the currently deployed public pool.

This paper specifies the economic model, claim encoding, pricing rule, collateral accounting, settlement semantics, agent interface, trust assumptions, and implementation status. It does not claim that market prices are true probabilities, that participation is profitable, or that the testnet mechanism is ready for real-value deployment.

Keywords: prediction markets, combinatorial markets, LMSR, joint probability, Boolean securities, automated market makers, variable elimination, autonomous agents

1. Problem statement

Let A and B be related binary events. A conventional interface may display a market for A and a separate market for B. A trader who has a view about the relationship between them must approximate that view using multiple trades, accept execution risk across legs, or avoid expressing it. More importantly, two separate markets do not determine P(A and B). Knowing P(A) and P(B) constrains the joint probability only through the Frechet bounds:

```
max(0, P(A) + P(B) - 1) <= P(A and B) <= min(P(A), P(B)).
```

Any value inside this interval can be consistent with the same marginals. Independence is only one possible assumption:

```
P(A and B) = P(A) P(B) if and only if A and B are independent.
```

For example, suppose P(A) = 0.40 and P(B) = 0.30. Then P(A and B) may lie anywhere from 0 to 0.30. The independent baseline is 0.12. A joint market price of 0.22 therefore expresses positive association relative to that baseline, while remaining within the probability bounds. It does not establish that either event causes the other.

Flurbo treats the complete state assignment as the primitive economic object. Individual and composed claims are projections of one state space. This approach is intended to provide three properties.

1. **Coherent expression.** A trader can buy a claim whose payout matches a Boolean thesis rather than reconstructing that thesis from disconnected legs.
2. **Shared price impact.** Every accepted trade changes one inventory and therefore affects every claim that overlaps the same terminal states.
3. **Unified settlement.** One resolved state settles all claims in a pool according to their published truth tables.

The design does not solve semantic market definition by mathematics alone. A coherent numerical state space can still describe vague or incompatible real-world questions. Event wording, observation sources, deadlines, and exception policies must be reviewed and committed before trading.

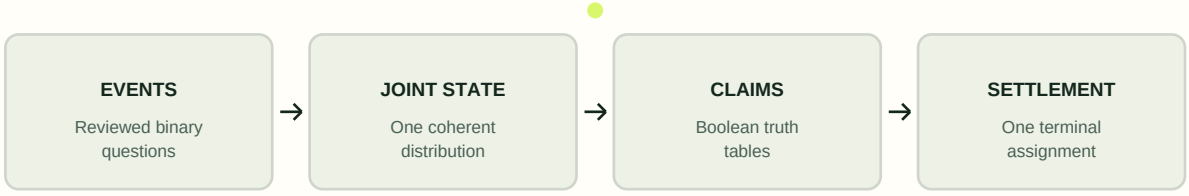


Figure 1. Flurbo maps reviewed events into one joint state space, prices Boolean claims from one inventory, and settles them with one terminal assignment.

2. State space and claim language

2.1 Binary events and terminal states

A pool contains n ordered binary events E_0, ..., E_(n-1). A terminal state x is an n-bit vector in {0,1}^n. Bit i equals one when event E_i resolves YES and zero when it resolves NO. The set of terminal states is Omega = {0,1}^n.

For two events A and B, the four states are:

0	No	No	Neither occurs
1	Yes	No	A only
2	No	Yes	B only
3	Yes	Yes	Both occur

2.2 Canonical Boolean claims

A claim is identified by a pair (S, M).

- S is a scope bitmap that selects one to three events in the current public claim interface.
- M is a nonempty, nonuniversal truth-table mask over the local assignments of the selected events.
- One unit of a claim pays one collateral unit in each local state selected by M, subject to the void rule in Section 6.

If k events are selected, the local state space has 2^k assignments and M has 2^k possible bits. The event order is the ascending order of set bits in S. Constant-zero and constant-one claims are excluded because they do not express a contingent outcome.

For two positive legs A and B, the principal named claims are:

A	A only, both	10	Individual YES on A
B	B only, both	12	Individual YES on B
A AND B	both	8	Pays only if both occur
A OR B	A only, B only, both	14	Pays if at least one occurs
Exactly one	A only, B only	6	Pays if one occurs and the other does not

For three selected legs, the named interface also includes exactly-one and at-least-two claims. The mask remains authoritative. A label is a readable interpretation of the mask, not a separate security definition.

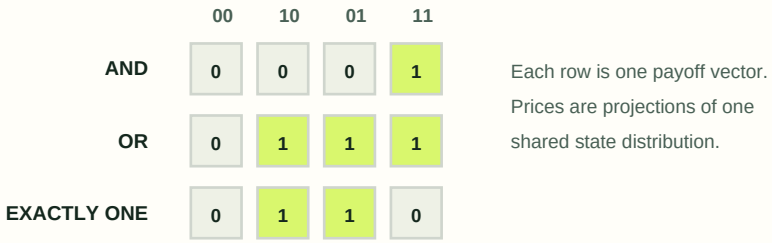


Figure 2. Boolean claims are payoff vectors over the same local states.

2.3 Scope and market boundaries

Claims can combine events only within the same pool. Two events displayed on the same website but assigned to different pool contracts do not share a state space, cost function, collateral reserve, or atomic combined claim. This boundary is essential. Flurbo does not infer a cross-pool joint distribution from unrelated marginal prices.

The public interface currently limits a composed claim to at most three events. The agent forecast representation supports normalized distributions over one to four events for research and policy evaluation. These limits are implementation bounds, not mathematical statements about the largest meaningful thesis.

3. Shared logarithmic pricing

3.1 Cost function

Let q_x denote the total outstanding payout liability in terminal state x. For liquidity parameter b > 0, the logarithmic market scoring rule cost function is

```
C(q) = b log( sum_(x in Omega) exp(q_x / b) ).
```

The instantaneous state price is the gradient of the cost function:

```
p_x(q) = exp(q_x / b) / sum_(y in Omega) exp(q_y / b).
```

The vector p(q) is nonnegative and sums to one. For any Boolean claim with winning set W, its marginal price before finite-size price impact is

```
p_W(q) = sum_(x in W) p_x(q).
```

The quoted collateral for buying delta units of the claim is the cost difference

```
Q_buy = C(q + delta 1_W) - C(q),
```

where 1_W is one in winning states and zero otherwise. A sale reverses liability and receives the corresponding decrease in C, subject to ownership and representational constraints.

Hanson introduced logarithmic market scoring rules as automated mechanisms for information aggregation [1, 2]. Combinatorial state spaces make exact LMSR pricing difficult in general, with known hardness results even under restricted languages [3]. Flurbo therefore maintains a small enumerated implementation for deployed testnet clusters and a separately bounded factored implementation for larger sparse clusters.

3.2 Finite trade formula

For a Boolean claim whose current probability is p, the ideal fee-free cost of buying delta units is

```
Q_buy = b log(1 + p (exp(delta / b) - 1)).
```

This expression is algebraically equal to the global cost difference. It also makes the role of b explicit. Larger b produces more depth and less price movement for a fixed quantity. Smaller b increases sensitivity and reduces the initial subsidy, but also produces larger slippage.

The deployed contracts use integer collateral atoms and conservative numerical bounds. Buy quotes round outward so that the pool collects at least the proven ideal cost increase. Sell quotes round outward so that the pool pays at most the proven ideal cost decrease. A quote is a snapshot rather than a reservation. Execution recomputes it and enforces a trader-supplied maximum cost or minimum proceeds together with a deadline.

3.3 Coherence and its limits

Because every claim price is derived from the same state-price vector, logical relations hold at each state snapshot. For example,

```
P(A OR B) = P(A) + P(B) - P(A AND B).
```

The mechanism therefore does not maintain independent order books for A, B, and A AND B. A trade in one expression changes the relative liability of states and can move all overlapping expressions.

This numerical coherence should not be overstated. Prices are inventory-derived market quantities. They can reflect sparse participation, subsidized initial conditions, strategic behavior, test-token indifference, or poor event definitions. They are not independently validated forecasts and they do not prove causation.

4. Bounded factored representation

4.1 Motivation

Explicit enumeration requires 2^n state liabilities. This is practical for small clusters and unsuitable for large n. The factored engine represents total liability as a sum of local tables:

```
q(x) = sum_f q_f(x_scope(f)).
```

The partition function is

```
Z = sum_x exp(q(x) / b),
```

and C(q) = b log Z. The factor potentials exp(q_f / b) multiply into one global distribution. The factors are not separate market makers and do not create separate pools of collateral.

4.2 Variable elimination

Flurbo's factored implementation evaluates log Z and maximum liability with variable elimination. A caller supplies a complete elimination order. The current Solidity domain accepts:

- 1 to 32 binary events;
- at most 64 factors;
- at most 3 events in each factor scope;
- induced width at most 2 for the supplied order;
- fixed numeric bounds relative to b.

Under width 2, joined intermediate tables contain at most eight entries. The evaluator never allocates a 2^n global state array. Exact max-sum elimination computes the largest outstanding terminal payout, while interval log-sum-exp arithmetic produces conservative cost bounds.

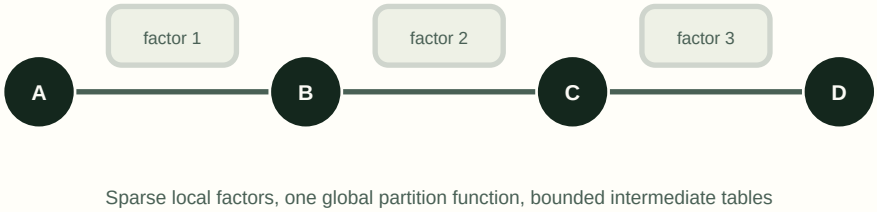


Figure 3. A bounded-width factor graph represents local interactions while preserving one global cost function.

The supplied order is validated but not optimized. A graph may admit a valid low-width order and still be rejected when given a poor order. Adding a claim can also create fill-in that violates the bound. These are intentional rejection conditions, not silent approximations.

4.3 Current implementation boundary

The factored Solidity pool integrates bounded-width evaluation, conservative quotes, owner holdings, funding, exact transfer checks, settlement, redemption, and canonical base-event receipt tokens. It has extensive local tests. As of this version, it is not the pool serving the public Flurbo testnet application. Public deployment, live gas validation, indexing, and operational acceptance remain future gates.

5. Collateral, ownership, and trading

5.1 Initial funding

With no outstanding claims and a uniform initial state, the LMSR cost is

$$C(0) = b \log |\Omega| = b n \log 2.$$

The pool is funded before trading with a conservative rounded amount derived from this initial cost. Funding supplies the market maker's bounded subsidy. The present contracts do not issue an LP token, provide a sponsor withdrawal right, or treat surplus as distributable profit.

5.2 Coverage invariant

Let L(x) be the payout owed in terminal state x. Before resolution, required collateral is

```
R = max_(x in Omega) L(x).
```

The pool checks that its actual collateral balance covers R. After resolution, required collateral becomes the remaining payout in the realized state. Conservative quoting and outward rounding are designed to preserve coverage, but the contracts also check the balance directly.

Coverage is a solvency property. It does not mean that every participant can profit, that subsidy is recoverable, or that a malicious collateral token is safe. Current implementations require exact transfer amounts and reject ordinary fee-on-transfer behavior.

5.3 Positions and sales

Holdings are keyed by owner and exact claim identity. A trader may sell only units of the same claim that the trader owns. Ownership of a logically related claim, a superset claim, or another account's position does not authorize a sale.

Base YES and NO claims may be wrapped into canonical ERC-20 receipt tokens. Wrapping transfers the internal holding into token escrow and mints an equal receipt balance. It does not create new liability, move collateral, or change prices. Composed claims remain internal pool positions in the current implementation.

Each trade is atomic at the contract boundary. State mutation, collateral transfer, ownership update, and post-trade coverage checks either complete together or revert together. Frontend review and agent policies add safeguards but do not replace contract-level slippage, deadline, and ownership checks.

6. Resolution, void outcomes, and redemption

6.1 Immutable rules

Each pool commits a nonzero hash of its settlement rules. The published preimage is expected to define ordered event identities, observation sources, observation windows, binary interpretation, finality conditions, and exception policy. The contract verifies the hash relationship and designated resolver behavior. It cannot determine whether the natural-language rule is useful, complete, or truthful.

The public pilot uses an optimistic testnet resolver. Assertions require test AUSD bonds. An unchallenged assertion can finalize after the challenge period. A challenge opens voting by a fixed reviewer panel. Failure to assert or reach quorum can produce VOID. The currently documented alpha panel is operator controlled, so it should be treated as centralized testnet resolution rather than independent adjudication.

For new holder-restricted deployments, challenge eligibility can require a current related holding linked to the same Mera account. Existing legacy resolvers do not retroactively obtain that restriction. Holder eligibility restricts access to the challenge function; it does not make the asserted outcome correct.

6.2 Uniform void semantics

For resolved YES or NO events, the corresponding bit is fixed. Each VOID event is averaged uniformly over its two possible binary answers. If V is the set of void events and K is the fixed partial assignment, the payout fraction of claim W is

```
phi(W | K,V) = |{x compatible with K : x in W}| / 2^|V|.
```

Thus a unit of A AND B pays 0.5 when A is YES and B is VOID, 0 when A is NO and B is VOID, and 0.25 when both events are VOID. VOID is not equivalent to NO and does not refund the purchase price. Redemption converts this

fraction into collateral atoms with conservative rounding.

6.3 Redemption

After all base outcomes are delivered, the owner redeems an exact claim and quantity. Winning or fractional payout is calculated from the claim truth table and the final YES and VOID masks. The redeemed holding is burned. Partial redemption is supported, and the remaining reserve continues to protect other holders.

Resolution risk remains material. An incorrect assertion that is not challenged can finalize. Reviewer collusion, unavailable infrastructure, ambiguous rules, missing evidence, or loss of an authorization service can impair the process. The current testnet system should not be interpreted as a decentralized oracle.

7. Analysis and thesis exploration

7.1 Joint forecasts

Flurbo distinguishes market prices from a user's private forecast. An agent or user forecast is represented as integer parts per million over every state of one to four selected events. The entries must be nonnegative and sum exactly to 1,000,000. Marginals, joints, conditionals, and claim probabilities are derived from this distribution without floating-point ambiguity.

For two events, a user can provide P(A), P(B), and P(A and B). Inputs outside the Frechet bounds are rejected. The resulting four-state distribution is

```
P(0,0) = 1 - P(A) - P(B) + P(A and B)
     P(1,0) = P(A) - P(A and B)
     P(0,1) = P(B) - P(A and B)
        P(1,1) = P(A and B).
```

The What-if and thesis interfaces compare such user estimates with pool-implied values and with an independence baseline. Conditional quantities describe association under a stated condition. They do not establish causal effects.

7.2 Expected value and executable quotes

If a claim pays one collateral unit and a user assigns it probability pi, then the user's expected gross payout for quantity delta is delta pi. A quote cost Q implies the user's subjective expected net value

```
EV = delta pi - Q - gas - external fees.
```

This expression is not a profit guarantee. The forecast may be wrong, the quote moves with size, execution may fail, and capital may remain locked until resolution. Flurbo displays executable quotes rather than treating displayed marginal probabilities as fixed purchase prices.

8. Autonomous agent architecture

Flurbo exposes a versioned runtime protocol and SDKs for TypeScript, Python, Rust, and C++. An agent runs on infrastructure controlled by its operator. A typical pipeline is:

```
research -> normalized joint forecast -> strategy -> policy checks
-> pool quote and simulation -> persistent intent -> local signature
-> broadcast -> receipt reconciliation -> verified attribution
```

API credentials authorize read and reporting operations. They are not signing keys. A dedicated agent wallet signs locally, and Flurbo does not receive its private key. Verified attribution requires that the reported transaction, sender, calldata, chain, receipt, and resulting pool event match the recorded intent.

The reference policy checks market allowlists, market close time, quantity, trade cost, slippage, quote age, total and rolling spend, position size, worst-case loss, gas fee, gas reserve, and the number of pending transactions. The

journal preserves planned, signed, broadcast, confirmed, reverted, replaced, expired, and mismatch states so a restart can reconcile uncertain submissions without blindly duplicating a transaction.

These controls are software policy in a compliant runtime. Possession of a raw externally owned account key can bypass them. Dashboard pause and credential revocation stop new actions by compliant runtimes; they do not cancel an already submitted transaction or remove the wallet's on-chain authority. Cryptographically enforced delegation would require a separate smart-account design.

The SDK validates each joint forecast and computes worst-case payout across pool states. A semantic compatibility layer can reject configured impossible claims before signing. This layer depends on reviewed observation mappings. It must abstain when the relationship is unknown rather than infer exclusivity from question text alone.

9. System architecture and operational controls

The hosted application separates four responsibilities.

1. **Browser application.** Presents public documentation, invite access, markets, thesis exploration, transaction review, portfolio history, settlement status, and agent management.
2. **Read and indexing services.** Batch block-pinned RPC reads, verify canonical checkpoints, index bounded history, and distinguish incomplete indexing from an empty account.
3. **Account and agent services.** Maintain passkey sessions, approved testnet access, linked trading wallets, credentials, agent journals, and verified transaction attribution.
4. **On-chain contracts.** Enforce pricing, ownership, collateral transfer, deadlines, settlement binding, and redemption.

Shared hosted capacity is protected by separate anonymous, account, and credential budgets, bounded RPC concurrency, queue limits, and cooldown responses. These controls reduce accidental or abusive load. They do not provide a service-level guarantee, and provider quotas remain external dependencies.

The public testnet uses Monad chain ID 10143 and test AUSD with six decimals. Test assets have no intended monetary value. Mera passkeys authenticate the Flurbo account, while MetaMask or a dedicated agent wallet signs trades. Account identity and transaction authority are intentionally separate.

10. Security and trust model

10.1 On-chain guarantees within the implementation domain

The reviewed contract design aims to enforce the following properties:

- one immutable collateral token, liquidity parameter, close time, resolver, and rules hash per pool;
- deterministic claim identity through scope and mask;
- quote recomputation at execution;
- inclusive slippage and deadline limits;
- exact owner debits and credits;
- reentrancy protection on mutable entry points;
- collateral coverage before and after value-moving operations;
- final settlement that is applied consistently to every claim truth table;
- replay-resistant transaction and agent attribution at the hosted boundary.

These are implementation objectives supported by tests, not a substitute for an independent audit or a formal proof of the complete deployed system.

10.2 Trusted and operational components

The system still depends on:

- accurate event definitions and publication of the committed rule preimage;
- the designated resolution process and its evidence sources;
- RPC availability and correct canonical-chain responses;
- durable hosted storage for sessions, evidence, indices, and agent records;
- users protecting passkeys and transaction-signing wallets;
- operators maintaining monitoring and settlement transactions;
- the collateral token preserving expected ERC-20 balance semantics.

The current reviewer configuration and infrastructure are unsuitable for claims of decentralized adjudication. The system also has no mechanism to guarantee that market participation supplies accurate information.

10.3 Economic risks

A user can lose the complete purchase cost of a claim. Combined claims usually have narrower winning sets than their individual legs and can therefore offer larger payout multiples together with higher loss probability. OR and threshold claims win in more states but generally cost more. These relationships depend on current inventory and trade size.

No combination is favorable merely because its events are correlated. An observable relationship is useful only when the trader's probability for the claim differs sufficiently from the executable quote after price impact, gas, and settlement risk. Buying mutually exclusive claims separately can guarantee that at least one leg loses, but it does not necessarily guarantee a negative portfolio payoff because quantities, costs, other positions, and void payouts matter. Risk analysis must evaluate the complete payout vector.

11. Implementation status

Public web application	Deployed testnet beta	Invite-based, test assets
Small-cluster PilotPool	Deployed on Monad testnet	Factored local scopes over a small event cluster
AND and OR claims	Implemented	Manual and SDK claim encoding
Exactly-one and at-least-two claims	Implemented in canonical claim model	Availability depends on market and UI context
Uniform VOID payout	Implemented in pilot settlement	Fractional truth-table averaging
Holder-restricted challenge resolver	Implemented for new deployments	Not retroactive to legacy resolvers
Agent runtime and SDKs	Implemented in four languages	Local keys, paper and testnet live modes
Bounded-width factored LMSR pool	Local implementation and tests	Not the current public production pool
Independent oracle or appeal system	Not implemented	Current alpha remains operator dependent
Mainnet or real-value market	Not launched	No real-value readiness claim
Independent security audit	Not completed	Required before real-value deployment

The deployed pilot documented in the repository uses pool address `0x28d5ee02b1eda6959ac3ee5a4f834237c84dee02` on Monad testnet. Addresses and active market configuration can change through new deployments, so the hosted application and its verified manifests remain the operational source for current testnet markets.

12. Limitations and research agenda

The principal limitations are as follows.

1. **Sparse-graph restriction.** The factored engine rejects updates that exceed the fixed induced-width bound. General combinatorial LMSR pricing remains computationally difficult [3].
2. **No automatic structure discovery.** The protocol does not infer conditional independence or semantic exclusivity from text. Operator-reviewed mappings are required.
3. **No general conditional security.** Current Boolean claims pay on terminal truth tables. A security such as B given A, with no exposure when A is false, requires a separately specified payoff and trading treatment.
4. **Conservative selling.** The factored representation requires exact-scope liability for sales. Logically equivalent claims on other scopes are not automatically netted.
5. **Resolution centralization.** The current alpha panel is operator controlled and a wrong uncontested assertion can finalize.
6. **Operational dependence.** Hosted storage, RPC providers, monitoring, and settlement workers remain availability dependencies.
7. **No proof of demand or forecast quality.** Technical coherence does not establish that users want the product or that prices outperform other forecasting methods.
8. **No real-value readiness.** Test coverage does not replace contract audit, economic review, incident response, or jurisdiction-specific legal analysis.

Future work includes automatic elimination-order selection, larger sparse clusters, conditional securities, portfolio-level coherent forecasts across all agent decisions, stronger on-chain delegation, independent resolution, production indexing, public verification tooling, gas optimization, and empirical evaluation of user behavior and forecast calibration.

13. Conclusion

Flurbo's central proposition is that related predictions should be represented as claims on one coherent outcome space. A shared cost function gives individual and Boolean claims mutually consistent prices, shares liquidity across overlapping states, and permits one terminal assignment to settle all claims. The small-cluster public testnet provides an operational environment for evaluating that proposition. The bounded-width factored engine provides a concrete route toward larger sparse event graphs without global state enumeration.

The remaining work is substantial. Semantic market design, resolution quality, operational reliability, computation, and user demand are separate problems from numerical coherence. The purpose of the testnet is to measure those problems directly while limiting financial risk to test assets.

Appendix A. Worked two-event example

Consider two events:

- A: a network upgrade launches within a specified window.
- B: a major exchange supports that upgrade within a specified window.

Suppose the pool-implied values before a finite trade are P(A) = 0.30, P(B) = 0.40, and P(A and B) = 0.22. The complete distribution is:

Neither	0.52	0	0	0
A only	0.08	0	1	1
B only	0.18	0	1	1
Both	0.22	1	1	0

The independence baseline for both is 0.30 x 0.40 = 0.12. The market-implied joint value is 0.10 higher, but this difference is an association measure only. The derived claim probabilities are:

```
P(A AND B) = 0.22
P(A OR B) = 0.30 + 0.40 - 0.22 = 0.48
P(exactly one) = 0.08 + 0.18 = 0.26.
```

A trader who assigns P(A and B) = 0.28 may consider an AND purchase, but must compare that belief with the finite executable quote. A displayed 0.22 marginal does not imply that a large order costs 0.22 per unit. The cost curve increases as the order shifts liability toward the winning state.

Appendix B. Claim mask construction

For a scope containing k ordered events, enumerate local states s from 0 to 2^k - 1. Evaluate the Boolean predicate g(s). Set bit s of M when g(s) = 1. The claim payout in a full pool state x is determined by projecting x onto the scope and testing the corresponding bit of M.

This procedure separates human-readable logic from execution representation. Contracts need only validate S and M, update each winning local liability, and apply the final projected state. SDKs must reproduce the same ordering and mask exactly.

References

[1] Hanson, R. (2003). Combinatorial information market design. *Information Systems Frontiers*, 5(1), 107-119. https://doi.org/10.1023/A:1022058209073

[2] Hanson, R. (2007). Logarithmic market scoring rules for modular combinatorial information aggregation. *Journal of Prediction Markets*, 1(1), 3-15. https://hanson.gmu.edu/mktscore.pdf

[3] Chen, Y., Fortnow, L., Lambert, N., Pennock, D. M., and Wortman, J. (2008). Complexity of combinatorial market makers. *Proceedings of the 9th ACM Conference on Electronic Commerce*, 190-199. https://doi.org/10.1145/1386790.1386822

[4] Pennock, D. M., and Xia, L. (2012). Price updating in combinatorial prediction markets with Bayesian networks. *Proceedings of the 28th Conference on Uncertainty in Artificial Intelligence*. https://doi.org/10.48550/arXiv.1202.3756

[5] Flurbo Research and Engineering. (2026). Repository specifications for quoting, factored pricing, settlement, account challenges, and autonomous agents. Version 0.1 code and documentation snapshot, 28 September 2026.

Document status

This document describes an experimental testnet system and selected locally tested research implementations. Test assets only. It is not investment advice, a promise of profit, an independent security audit, or a claim of legal approval. The source document and build script are maintained with the application repository so changes can be reviewed alongside code.